

cutlass

kernel 调用逻辑:

cutlass/gemm/dispatch.h

```
return dispatch<math_op, block_task_policy_t, TransformA, LdgAlignA, TransformB,
LdgAlignB, value_t, accum_t, epilogue_op_t, LdgAlignC, AllowRaggedTiles>(
    kernel<math_op, block_task_policy_t, TransformA, LdgAlignA, TransformB, LdgAlignB,
value_t, accum_t, epilogue_op_t, LdgAlignC, AllowRaggedTiles>,
    m,
    n,
    k,
    epilogue_op,
    d_a,
    d_b,
    d_c,
    stream,
    debug_synchronous);
```

kernel函数入口:

cutlass/gemm/dispatch.h

```
__global__ void kernel(param_pack<value_t, accum_t, epilogue_op_t> pack)
{
    // Parameterize task type
    typedef typename gemm_block_task<
        math_op,
        block_task_policy_t,
        value_t,
        accum_t,
        TransformA,
        LdgAlignA,
        TransformB,
        LdgAlignB,
        epilogue_op_t,
        LdgAlignC,
        AllowRaggedTiles>::type block_task_t;

    // Declare statically-allocated shared storage
    // 唯一一个声明共享内存的地方
    __shared__ typename block_task_t::scratch_storage_t smem;

    // Construct and run the task
    block_task_t(
        &smem,
        pack.d_a,
        pack.d_b,
```

```

    pack.d_c,
    pack.epilogue_op,
    pack.m,
    pack.n,
    pack.k,
    pack.k_split).run();
}

```

计算逻辑执行run()函数:

cutlass/gemm/block_task.h

```

void run()
{
    // Quit if the thread block is fully out-of-bounds
    // 线程超出范围直接退出
    if (grid_raster.is_block_oob(dim_m, dim_n))
    {
        asm volatile("exit;");
    }

    // Request global prefetch of first tile
    // 对全局内存的预取, 预取到寄存器
    loader_a.request();
    loader_a.next();
    loader_b.request();
    loader_b.next();

    // Commit global prefetch of first tile to shared memory
    // 从寄存器取数据到共享内存
    loader_a.commit(scratch->pages[page_idx].block_a);
    loader_b.commit(scratch->pages[page_idx].block_b);

    // Advance to next A,B tiles in K-axis
    block_item_coords_k += BlockItemsK;

    // Synchronize shared tiles and prepared accumulator
    __syncthreads();

    // Initialize thread's slice of accumulators
    accumulator.init();

    // Request first iteration of local prefetch strips
    request_local_prefetch(
        local_slices_a[0],
        local_slices_b[0],
        0);

    //
    // Main loop

```

```

//

// Consume tiles in A and B along the K-axis (all but last tile)
#pragma unroll 1
while (block_item_coords_k < block_end_item_k)
{
    consume_tile<true>();

    // Advance to next A,B tiles in K-axis
    block_item_coords_k += BlockItemsK;
}

// Consume last tile
consume_tile<false>();

//
// Epilogue
//

epilogue();
}

```

核心逻辑:

```

/**
 * Consume a tile of A and B each
 */
template <bool DoGlobalPrefetch>
inline __device__
void consume_tile()
{
    // Unroll BlockDpVectorsK iterations of outer-product accumulations
    // 循环展开
    #pragma unroll
    for (int tile_offset_k = 0; tile_offset_k < BlockDpVectorsK; tile_offset_k += 1)
    {
        // Last strip commits global prefetch for next tile
        if ((tile_offset_k == BlockDpVectorsK - 1) && DoGlobalPrefetch)
        {
            // If not using two pages of scratch tiles, protect the above prefetch
            // loads from the committing writes below
            if (!UseDoubleScratchTiles)
                __syncthreads();

            // If using two pages of scratch tiles, switch to next page before
            // writing
            if (UseDoubleScratchTiles)
            {
                page_idx = (page_idx ? 0 : 1);
            }
        }
    }
}

```

```

        // Commit global prefetch data to scratch page
        loader_a.commit(scratch->pages[page_idx].block_a);
        loader_b.commit(scratch->pages[page_idx].block_b);

        __syncthreads();
    }

    // Request local prefetch for next strip
    // 这里加载的是下一个循环要用到的数据
    request_local_prefetch(
        local_slices_a[(tile_offset_k + 1) % 2],
        local_slices_b[(tile_offset_k + 1) % 2],
        (tile_offset_k + 1) % BlockDpVectorsK);

    // Request global prefetch for next tile on first strip
    if ((tile_offset_k == 0) && DoGlobalPrefetch)
    {
        loader_b.request();
        loader_b.next();
        loader_a.request();
        loader_a.next();
    }

    // Cast strip-mined loads to contiguous array of dp_vector_t
    typedef dp_vector_t thread_tile_a_t[ThreadLdsVectorsA * LdsVectorDpVectorsA];
    typedef dp_vector_t thread_tile_b_t[ThreadLdsVectorsB * LdsVectorDpVectorsB];
    thread_tile_a_t &thread_tile_a = reinterpret_cast<thread_tile_a_t&>
(local_slices_a[(tile_offset_k) % 2]);
    thread_tile_b_t &thread_tile_b = reinterpret_cast<thread_tile_b_t&>
(local_slices_b[(tile_offset_k) % 2]);

    // Accumulate this dp-stripe product
    accumulator.multiply_accumulate(thread_tile_a, thread_tile_b);
    // request_local_prefetch 和 multiply_accumulate 分别负责取数据和计算，在数据流图上
    没有依赖，并且使用的是不同的硬件单元，在指令级别是并行执行的。
    }
}

```

计算逻辑：

```

/**
 * \brief Compute the product of tile_a and tile_b and add the result to
 * the tile of accumulators.
 */
inline __device__
void multiply_accumulate(
    dp_vector_t (&tile_a)[ThreadItemsY],
    dp_vector_t (&tile_b)[ThreadItemsX])
{

```

```

    // Simply traverse the accumulator tile in row-major order
    #pragma unroll
    for (int y = 0; y < ThreadItemsY; ++y)
    {
        #pragma unroll
        for (int x = 0; x < ThreadItemsX; ++x)
        {
            mad_xy(tile_a, tile_b, x, y);
        }
    }
}

```

以f32为例，调用的PTX指令执行的计算:

```

/// Compute "dp1" float->float
inline __device__
static void mad(
    float &d,
    const float &a,
    const float &b,
    const float &c)
{
    asm volatile ( "fma.rn.f32 %0, %1, %2, %3;\n"
        : "=f"(d) : "f"(a), "f"(b), "f"(c));
}

```

V100上使用tensor core的计算逻辑

```

/**
 * \brief Compute the product of tile_a and tile_b and add the result to
 * the tile of accumulators.
 */
inline __device__
void multiply_accumulate(
    fragment_a_t (&tile_a)[WmmaBlocksY],
    fragment_b_t (&tile_b)[WmmaBlocksX])
{
    #pragma unroll
    for (int x = 0; x < WmmaBlocksX; ++x)
    {
        #pragma unroll
        for (int y = 0; y < WmmaBlocksY; ++y)
        {
            nvcuda::wmma::mma_sync(accumulators[x][y], tile_a[y], tile_b[x],
accumulators[x][y]);
        }
    }
}

```

```
}
```

- 时钟周期

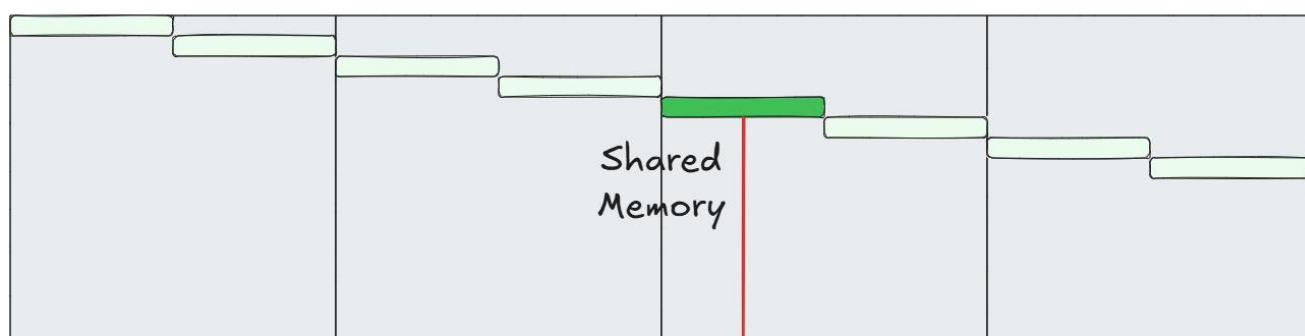
操作	延迟 (Latency)	吞吐 (Throughput)
FMA FP32	4 cycles (Ampere)	1 instr / cycle / warp (每 SM 通常 16~32 op/cycle)
INT MUL/ADD	4 cycles	1 instr / cycle / warp
寄存器Reg → Reg	1 cycle (几乎无延迟)	
寄存器间通信	≤ 1 cycle	
共享内存-Shared → Reg (ld)	1~32 cycles (没有bank冲突是1)	float x = smem[threadIdx.x]; 需要通过地址解析、bank arbitration(存储体仲裁);
L1缓存	1~32	
L2缓存	32~64	
全局内存	400~600	
CUDA Graphs	约 500-1000 个时钟周期	
传统 Launch	通常超过 2000 个时钟周期 (含 CPU-GPU 通信开销)	

- 多发射、指令乱序：对于数据没有依赖的场景下，加载数据与计算同时进行
- 分块: **hreadBlock-Warp-Thread**三级分块: BlockTile（线程块级）、WarpTile（warp级）和ThreadTile（线程级）
- 全局内存的内存事务：wrap访问相邻的全局内存合并内存事务
- 共享内存的banck冲突：
swizzle减少**Bank**冲突: shared memory 的列偏移可以由行偏移的若干位与 **Global Memory** 中的列偏移异或得到



addr (4, 0:7): 0b 0100 000 0000 (1111)

行偏移 16行 列偏移 $64/8 = 8$ 列 块内字节偏移 $8 \times 2B = 16B$



addr (4, 32:39): 0b 0100 100 0000 (1111)

- 寄存器的bank冲突:

Volta/Turing/Ampere 架构, register files (RF) 是有两组sram Banks,每组Bank的位宽是64bit。

以一个FMMA指令为例, "FMMA R6, R97, R99, RX", 只要三个源寄存器不会同时访问同一个Bank就不会出现寄存器访问冲突。

- BlockSwizzling:** 相邻SM处理的线程块在数据空间上也相邻, 提升L2缓存命中率

A是row-major, B是col-major, C是row-major:

NVIDIA AMPERE ARCHITECTURE - TENSOR CORE OPERATIONS

PTX	Data Types (A * B + C)	Shape	Speedup on NVIDIA A100 (vs F32 CUDA cores)	Speedup on Turing* (vs F32 Cores)	Speedup on Volta* (vs F32 Cores)
mma.sync.m16n8k16 mma.sync.m16n8k8	F16 * F16 + F16 F16 * F16 + F32 BF16 * BF16 + F32	16-by-8-by-16 16-by-8-by-8	16x	8x	8x
mma.sync.m16n8k8	TF32 * TF32 + F32	16-by-8-by-8	8x	N/A	N/A
mma.sync.m8n8k4	F64 * F64 + F64	8-by-8-by-4	2x	N/A	N/A
mma.sync.m16n8k32 mma.sync.m8n8k16	S8 * S8 + S32	16-by-8-by-32 8-by-8-by-16	32x	16x	N/A
mma.sync.m16n8k64	S4 * S4 + S32	16-by-8-by-64	64x	32x	N/A
mma.sync.m16n8k256	B1 ^ B1 + S32	16-by-8-by-256	256x	128x	N/A

<https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#warp-level-matrix-instructions-mma-and-friends>

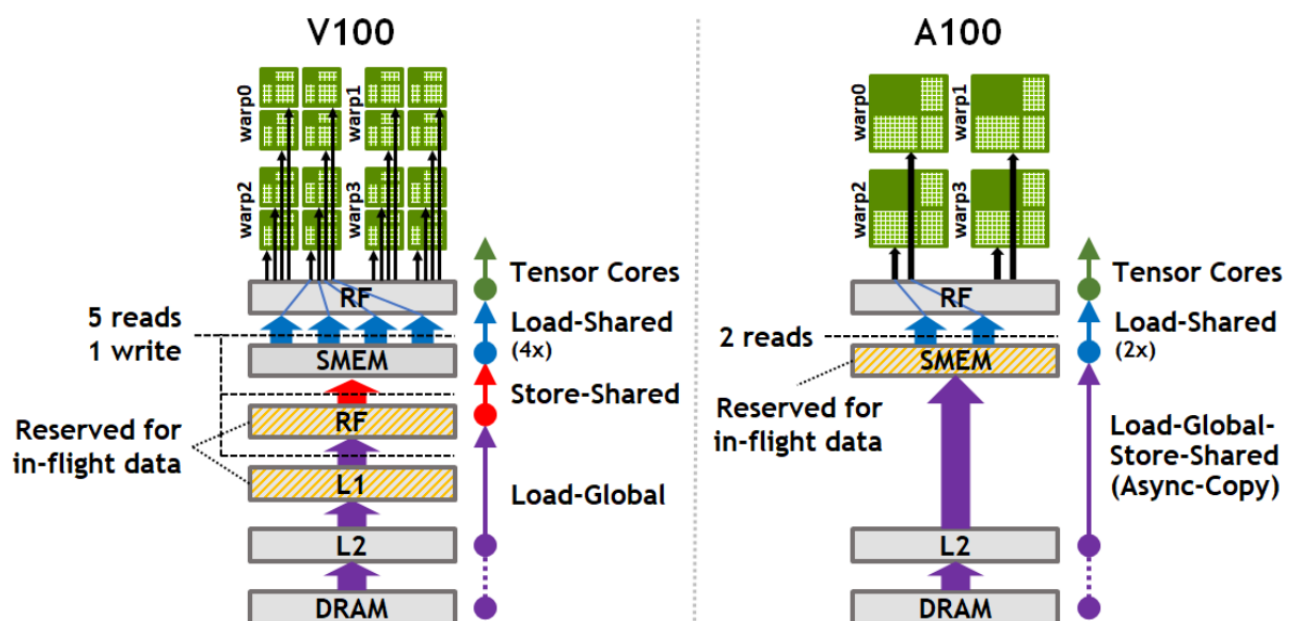
* Instructions with equivalent functionality for Turing and Volta differ in shape from the NVIDIA Ampere Architecture in several cases.

- A100 之前，使用 SMEM（shared memory）时，数据流是 Global memory -> Register -> Shared Memory，

必须要过一次 Register，效率很低。

A100 新增 LDGSTS（Load Global Storage Shared）指令，实现异步拷贝，GMEM -> SMEM，可不经 register file。

A100 SM Data Movement Efficiency 3x SMEM/L1 bandwidth, 2x in-flight capacity



A100 improves SM bandwidth efficiency with a new load-global-store-shared asynchronous copy instruction that bypasses L1 cache and register file (RF). Additionally, A100's more efficient Tensor Cores reduce shared memory (SMEM) loads.

- Ampere PTX指令

```
// ldmatrix
// 计算地址：线程组0-7加载第一个8x8矩阵，8-15加载第二个，以此类推
// 将通用指针（generic pointer）转换为指向共享内存（shared memory）的特定地址空间的指针
uint32_t offset = sizeof(uint16_t) * ((tid % 8) * 8 + (tid / 8) * 64); // 每组偏移64元素
uint32_t address = __cvta_generic_to_shared(smem) + offset;

// 加载4个8x8矩阵到寄存器（每个线程加载4个32位寄存器）
uint32_t frag[4];
if (tid < 32) { // 确保所有32线程参与
    asm volatile(
        "ldmatrix.sync.aligned.m8n8.x4.shared.b16 {%0, %1, %2, %3}, [%4];"
        : "=r"(frag[0]), "=r"(frag[1]), "=r"(frag[2]), "=r"(frag[3])
        : "r"(address)
        );
}
```

<https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=warp%2520level%2520matrix%2520instructions%2520mma%2520and%2520friends#warp-level-matrix-instructions-ldmatrix>

mma

<https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=warp%2520level%2520matrix%2520instructions%2520mma%2520and%2520friends#warp-level-matrix-fragment-mma-884-f16>

TODO:

- Stream-K Split-K
- cutlass 3.0 迭代器 bank 冲突
- wgmma

```
cutlass::gemm::GemmShape<128, 256, 8>, // BlockTile
cutlass::gemm::GemmShape<64, 64, 8>, // WarpTile
cutlass::gemm::GemmShape<8, 4, 8> // ThreadTile
```

- kernel fusion 例子: 余弦相似度
- DeepGemm
- Flash Attention
- cute DSL